

Code Refactoring with GitHub Copilot

- Why refactoring carries a different risk than generating new code
- Finding what's actually worth fixing
- Refactoring safely, one verified step at a time
- Folding a security fix into the same discipline
- Proving the improvement, and making it a team habit



Finding What's Actually Worth Fixing

- Ask Copilot to identify code smells and anti-patterns before changing anything
- Duplicated logic, an overly long conditional, a poorly named variable, a class doing too much
- The same context-aware reading from Module 2, now aimed at diagnosis rather than generation



Code Smells to Look For

Duplicated logic — Overly long conditionals — Poorly named variables — A class doing too much

Problem — A Wrong Refactor Hides in Plain Sight

- A wrong generation is an annoying first draft you simply don't accept
- A wrong refactor can silently change behavior that already works
- It can look like a clean improvement while breaking something real
- This is why refactoring gets its own discipline, not just Module 1's habit repeated

New code, wrong?
Just don't accept it.



Easy to Undo

Refactor, wrong?
May look like a clean improvement.



Silent Regression

Solution — Incremental, Reviewable Change

- One extracted method, one renamed variable, one simplified condition — at a time
- Verify (run tests, read the diff) between each step
- Not “clean up this whole file” reviewed as one enormous diff
- The same discipline scales up to legacy modernization and larger structural changes



Repeat, one small step at a time.

Refactoring with Security in Mind, Specifically

- A structural cleanup and a security fix are often the same motion
- Fixing an injection risk or a missing input check is a refactoring task, same as extracting a method
- Tightening input validation and sanitization, incrementally, verified at each step
- Named here, not only in Module 5 — “accept the whole diff” is most tempting exactly here



Same Discipline Applies

An unchecked field or an unsanitized value — add a specific guard, review the diff the same way as any other step.

HANDS-ON Incremental Refactor with a Security Fix

Proving It, and Making It a Team Habit

- A refactor is only finished once you can show the improvement, not just assert it
- A before/after comparison — readability, duplication, a relevant quality metric
- Agree as a team on the one-fix-at-a-time discipline before a deadline makes shortcuts tempting



Before

Messy, under-tested, hard to explain



After

Smaller, readable, defensively written